Enhancing Code Vulnerability Detection via Vulnerability-Preserving Data Augmentation

Shangqing Liu
Nanyang Technological University
Singapore
liu.shangqing@ntu.edu.sg

Wei Ma*
Nanyang Technological University
Singapore
ma_wei@ntu.edu.sg

Jian Wang
Nanyang Technological University
Singapore
jian004@e.ntu.edu.sg

Xiaofei Xie
Singapore Management University
Singapore
xfxie@smu.edu.sg

Ruitao Feng
Singapore Management University
Singapore
rtfeng@smu.edu.sg

Yang Liu
Nanyang Technological University
Singapore
yangliu@ntu.edu.sg

Abstract

Source code vulnerability detection aims to identify inherent vulnerabilities to safeguard software systems from potential attacks. Many prior studies overlook diverse vulnerability characteristics, simplifying the problem into a binary (0-1) classification task for example determining whether it is vulnerable or not. This poses a challenge for a single deep-learning based model to effectively learn the wide array of vulnerability characteristics. Furthermore, due to the challenges associated with collecting large-scale vulnerability data, these detectors often overfit limited training datasets, resulting in lower model generalization performance.

To address the aforementioned challenges, in this work, we introduce a fine-grained vulnerability detector namely *FGVulDet*. Unlike previous approaches, *FGVulDet* employs multiple classifiers to discern characteristics of various vulnerability types and combines their outputs to identify the specific type of vulnerability. Each classifier is designed to learn type-specific vulnerability semantics. Additionally, to address the scarcity of data for some vulnerability types and enhance data diversity for learning better vulnerability semantics, we propose a novel vulnerability-preserving data augmentation technique to augment the number of vulnerabilities. Taking inspiration from recent advancements in graph neural networks for learning program semantics, we incorporate a Gated Graph Neural Network (GGNN) and extend it to an edge-aware GGNN to capture edge-type information. *FGVulDet* is trained on a large-scale dataset from GitHub, encompassing five different types of vulnerabilities. Extensive experiments compared with static-analysis-based approaches and learning-based approaches have demonstrated the effectiveness of *FGVulDet*.

CCS Concepts: • **Security and privacy → Software security engineering**.

Keywords: Vulnerability Detection, Graph Neural Networks.

*Corresponding author

LCTES '24, June 24, 2024, Copenhagen, Denmark

1 Introduction

Software vulnerability is defined as a weakness in the software system that could be exploited by a threat source. With the increasing number of open-source libraries and the expanding size of software systems, the count of software vulnerabilities has been rising rapidly. Since these vulnerabilities can be exploited by malicious attackers, causing significant financial and social damages, vulnerability detection and patching have garnered widespread attention from academia and industry. For instance, the Common Vulnerabilities and Exposures (CVE) Program and the National Vulnerability Database (NVD) have been established to identify and patch vulnerabilities before they are exploited. So far, over 100,000 vulnerabilities have been indexed. However, in contrast to the quantity of open-source projects and the speed of software iteration, the number of exploited vulnerabilities is insufficient. In other words, there exists a large number of "silent" vulnerabilities that have not been exploited.

Automated software vulnerability detection remains a crucial yet far from the settled problem. Several techniques have been developed to detect vulnerabilities including static analysis [11, 43, 45], fuzzing [6, 47], symbolic execution [2, 4, 42].

Static analysis for vulnerability detection aims to identify vulnerabilities in the source code without execution, typically requiring substantial manual effort from security experts to craft rules. This approach has limited generalization ability across diverse vulnerabilities. Dynamic techniques, such as fuzzing and symbolic execution, identify vulnerabilities by dynamically executing programs. Dynamic approaches demonstrate relatively high precision in vulnerability detection, but configuring execution is complex, and execution results may be incomplete since not every program path can be executed.

Due to the capability of deep learning-based techniques to automatically extract features, more research focuses on utilizing DL techniques for vulnerability detection [9, 28, 29, 31, 40, 52, 54]. In early DL-based vulnerability detection works, some works [40] employed convolutional neural networks (CNNs) to leverage their powerful convolution capabilities for learning vulnerability-related features. However, as programs are not fixed length compared to images, they are not well-suited for CNNs. To avoid this problem, some other works [9, 29, 31, 54] treat programs as a flat sequence and apply recurrent neural networks (RNNs) with Long Short-Term Memory (LSTM) directly to learn the vulnerability features. Yet, in vulnerability scenarios, certain types of vulnerabilities, such as buffer overflow, are related to data flow, which cannot be captured by the program text alone. To capture the data dependency and control dependency of programs, Li et al. [28] proposed a program slicing algorithm based on the program dependency graph (PDG) to slice related statements and feed them to Bidirectional RNNs for learning. However, it still fundamentally treats programs as sequences.

How to learn well-structured control and data dependencies in programs? Devign [52] proposed an effective way by encoding programs into a code property graph (CPG) and utilizing this graph through GGNN [27] for vulnerability detection, achieving state-of-the-art performance. Afterward, there is a great number of works using GNNs to learn program semantics for source code vulnerability detection [7, 38, 46, 46]. However, most of these works combined various types of vulnerabilities to train a single classifier for vulnerability detection. Moreover, data augmentation has been shown to significantly improve performance on image data [18, 20, 21]. Recent works [23, 34] propose to augment code with the same functionality variants by the transformations for contrastive pre-training to learn code functionality for different downstream tasks. However, the defined transformations are at the granularity of the function and it cannot guarantee vulnerability-preserving attribution for vulnerabilities when transforming a function to the variants. Hence, how to perform data augmentation meanwhile preserving the source code vulnerability is a challenge.

To address these challenges, in this paper, we propose *FGVulDet*, which is a fine-grained vulnerability detector. Specifically, we train multiple classifiers via the enhanced GGNN for each type of vulnerability on the real collected vulnerability data set from GitHub. Then each model prediction result is ensembled for voting to give the final prediction. Furthermore, we propose a novel vulnerability-preserving data augmentation to enrich the diversity of data and improve the prediction performance. On the side of GGNN, we adopt it and further encode the edge type information along with the node features during message passing i.e., edge-aware GGNN for the enhancement of the vulnerability detection. We claim that the edge type information i.e., "Flow to", "Control" represents different semantics of programs, and encoding them explicitly during the learning process can facilitate learning more accurate code representations. An extensive evaluation is conducted on five different types of vulnerabilities compared with some static-analysis tools and deep-learning based vulnerability detection approaches have confirmed the superiority of our proposed approach. Further ablation study also reveals the effectiveness of each component in *FGVulDet*. Our contributions are as follows.

- We propose a novel vulnerability-preserving data augmentation technique to enrich the amount of the collected data and mitigate the limitations of rare vulnerabilities in quantity.
- We adopt an edge-aware GGNN by incorporating edge-type features with node features to improve the learning capacity of GGNN for vulnerability detection.
- We conduct extensive experiments on the real collected vulnerability data to illustrate the effectiveness of *FGVulDet*.

2 Background

2.1 Problem Definition

Existing works [12, 30, 40, 52] define source code vulnerability identification as a binary {0,1} classification problem i.e., labelling all vulnerable functions as 1, regardless of the vulnerable type of the function, which is coarse for vulnerability detection. Differently, in this work, we focus on investigating a fine-grained vulnerability identification problem i.e., for different types of vulnerability, our goal is to learn the corresponding prediction function. Specifically, given a dataset $D = \{D_1, D_2, ..., D_t\}$, where D_t is the sub-dataset in D for the vulnerability type $t \in T$ and T is a set of source code vulnerability types, we aim at learning the mapping $f_t \in F$ over D_t to predict whether the function has the vulnerability of type t and $F = \{f_1, f_2, ..., f_t\}$ is the prediction function for different vulnerability type. Furthermore, $D_t = \{(c_t, y) | c_t \in C_t, y \in \mathcal{Y}\}$, where C_t is a set of functions which contains the vulnerable functions with the vulnerability type t and the corresponding fixed functions and $y = \{0, 1\}$ is the label set with 1 for the vulnerability and 0 for the non-vulnerability.

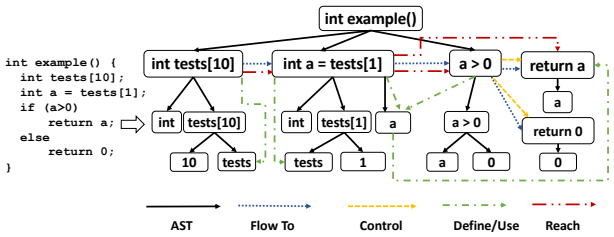


Figure 1. An example to illustrate Code Property Graph.

2.2 Code Property Graph

Code property graph (CPG) proposed by Yamaguchi et al. [50], combines several program representations e.g., Abstract Syntax Tree (AST), Control Flow Graph (CFG), Program Dependency Graph (PDG) into a joint graph to represent a program. An illustrated example is shown in Figure 1. We can observe that AST nodes (defined as black arrows in the graph) are the backstone of CPG. Besides AST, some other semantic representations i.e., control flow, and program dependency information can also be constructed on AST to represent different semantics of the program. For example, CFG represents the statement execution order of the program, and "Flow To" (blue arrow) represents this flow order in CPG. Furthermore, PDG is also involved in CPG, and the edges "Define/Use" (green arrow), "Reach" (red arrow) define the data dependencies, and the "Control"(yellow arrow) is the control dependency of a program.

2.3 Graph Neural Networks

Graph Neural Networks (GNNs) [25, 27] have been widely employed in modeling non-Euclidean data structure such as social networks [19, 25], protein-protein interaction networks [36]. The primary objective of a GNN is to identify patterns in graph data, relying on information within the nodes and their interconnectedness. There exist various GNN variants, here we describe the broad category of message-passing neural networks [17]. Suppose the original data can be modelled by a multi-edged graph, denoted as $G = (\mathcal{N}, \mathcal{E})$, where $\mathcal{N} = \{n_i\}$ is the node set and $\mathcal{E}$ is a set of directed edges $n_i \xrightarrow{k} n_j$ and k is the edge type. Each node n_i is endowed with vector representation $\boldsymbol{h}^t_{n_i}$ indexed over a timestep (hop) namely t. The node states are updated as

$$\boldsymbol{h}^{t+1}_{n_i} = f_t\left(\boldsymbol{h}^t_{n_i}, \bigoplus_{\forall n_j : n_i \xrightarrow{k} n_j} \left(m^t(\boldsymbol{h}^t_{n_i}, k, \boldsymbol{h}^t_{n_j})\right)\right) \tag{1}$$

where $m^t(\cdot)$ is a function that computes the message based on the edge label k. $\oplus$ is an aggregation operator that summarises the message from its neighbors and f_t is the update function that updates the state of node n_i. The initial state of each node $\boldsymbol{h}^0_{n_i}$ is from node-level information. Equation 1 updates all node states in a total number of T times recursively and at the end of this iteration, each $\boldsymbol{h}^T_{n_i}$ represents information about the node and how it belongs with the context of the graph. The well-known Graph Convolution Network(GCN) [25], Gated Graph Neural Network(GGNN) [27] also follow Equation 1, but the definitions of f_t and $m^t(\cdot)$ are different. For example, GGNN, which has been widely used in modeling source code [1, 32, 33, 52], employs a single GRU cell [8] for f_t, i.e., $f_t = GRU(\cdot,\cdot)$, $\oplus$ is a summation operation and $m^t(\boldsymbol{h}^t_{n_i}, k, \boldsymbol{h}^t_{n_j}) = \boldsymbol{E}_k \boldsymbol{h}^t_{n_j}$, where $\boldsymbol{E}_k$ is a learned matrix. The difference between GCN and GGNN lies in f_t is ReLU function [37] and $\boldsymbol{h}^{t+1}_{n_i}$ can be expressed as following equation:

$$\boldsymbol{h}^{t+1}_{n_i} = \mathrm{ReLU}\left(\boldsymbol{E}_t(\boldsymbol{h}^t_{n_i} + \sum_{\forall n_j : n_i \to n_j} \boldsymbol{h}^t_{n_j})\right) \tag{2}$$

3 Approach

3.1 Overview

The framework of our approach is illustrated in Figure 2, comprising three main components: Data Collection, which constructs a raw dataset with various types of vulnerabilities from commits; Vulnerability-preserving Data Augmentation, which enhances the original dataset with five mutation operations using a carefully designed vulnerability-preserving slicing algorithm to maintain the original vulnerability semantics and diversify the data; Edge-aware GGNN, which extends the current state-of-the-art GGNN by integrating edge type features into node features during message passing for model learning. During model training, we train multiple binary classifiers for different types of vulnerabilities. In the prediction phase, each classifier provides a prediction result, and *FGVulDet* aggregates their results through voting to obtain the final prediction.

3.2 Data Collection

Collecting high-quality datasets of vulnerable functions, especially encompassing various types of vulnerabilities, poses a significant challenge that necessitates expertise. In this work, we propose an effective method for collecting and labeling diverse types of vulnerable and non-vulnerable data. The process involves first gathering commits related to vulnerabilities, followed by extracting pairs of functions from these commits: the vulnerable version f_v and the patched version f_p, representing vulnerable and non-vulnerable functions, respectively. The detailed procedures are as below.

3.2.1 Vulnerability-Related Commit Collection. To assemble a sizable and diverse dataset of vulnerable functions, we initiate the process by gathering commits from 1614 C-language open-source projects hosted on GitHub. These projects are chosen for their popularity among developers

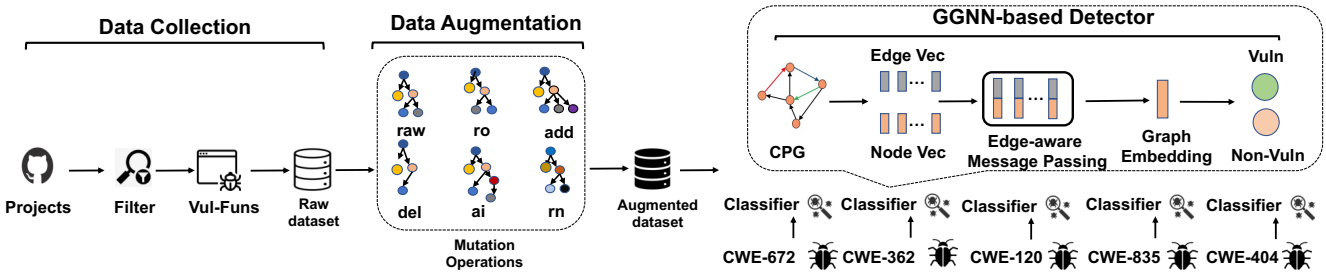


Figure 2. The framework of *FGVulDet*.

and their diversity in functionality, spanning various domains such as operating systems, networking, and database applications (e.g., Linux Kernel, OpenSSL, QEMU).

To ensure the quality of data labelling, we follow three steps to collect vulnerability-related Commits.

- *Commit Filtering.* We employ vulnerability-related keywords (shown in Table 1), which have been analyzed and summarized by a team of professional security researchers from a large number of commits. These keywords include five Common Weakness Enumerations (CWE) defined in the National Vulnerability Database (NVD), with each vulnerability type having one or more associated keywords. Commits whose messages do not match any of the keywords in Table 1 are excluded, and the remaining commits are considered more likely to be related to vulnerabilities. For example, in Figure 3, the vulnerability-related commit is accurately captured by the keyword
- *Type Matching.* Commits matched by keywords of multiple vulnerability types are excluded, as we cannot determine which vulnerability type they belong to. We retain commits matched by a single vulnerability type and use that type to label the commits.
- *Commit Pruning.* There are some vulnerability-related commits that may modify multiple functions, and not all of these functions are related to the vulnerability. We cannot automatically identify which function is related to the vulnerability, to alleviate this problem, we exclude those commits that modify more than one function. After the above three steps, we obtain a high-quality commit data set with vulnerability type labels.

3.2.2 Vulnerable/Non-vulnerable Function Extraction. Given the vulnerability-related commit as input, we can get its corresponding security patch P_v. We extract vulnerable functions f_v and patched functions f_p based on the change statements (i.e., the added statements S_{add} and the deleted statements S_{del}) from P_v. In this work, we take f_v as vulnerable functions and f_p as non-vulnerable functions. We can get a tuple (f_v, f_p, S_{del}, S_{add}), where S_{del}, S_{add} will be utilized for augmentation (See Section 3.3).

Table 1. Keywords of Five Vulnerability Types.

CWE	Vulnerability	Keywords
CWE-404	*Memory Leak*	memory leak, information leak, info leak, leak info, memory disclosure, leak memory, leak information
CWE-835	*Infinite Loop*	infinite loop, endless loop, long loop, infinite recursion, deep recursion
CWE-120	*Buffer Overflow*	buffer overflow
CWE-672	*Operation After Free*	double free, double-free, DF, use after free, use-after-free, UAF
CWE-362	*Race Conditions*	race conditions

```
Commit Message:  Fixed buffer overflow spotted by Henrik.
1  diff --git a/amixer/amixer.c b/amixer/amixer.c
2  index 646f0a9..e938cde 100644
3  --- a/amixer/amixer.c
4  +++ b/amixer/amixer.c
5   static char *simple_name(const char *name, char *result)
6   {
7  -    strncpy(result, name, simple_name_size);
8  -    result[simple_name_size] = '\0';
9  +    strncpy(result, name, simple_name_size - 1);
10 +    result[simple_name_size - 1] = '\0';
11      return result;
12  }
```

Figure 3. Patch for Buffer Overflow.

An illustrative example of a security patch is shown in Figure 3, we can get the changed statements S_{del} and S_{add} at line 7 to line 8 and line 9 to line 10, respectively. The vulnerable function f_v is composed from line 5 to line 8 and line 11 to 12 in Figure 3, and the patched function f_p is composed from line 5 to line 6 and line 9 to line 12.

3.3 Vulnerability-preserving Data Augmentation

As we collect different types of vulnerability data, it is difficult to ensure each type has a sufficient number for models to learn, hence we propose a data augmentation technique to scale up the collected data D in Section 3.2. Furthermore, the newly generated data must retain the vulnerability characteristics of the original data, i.e., it needs to be vulnerability-preserving. If the vulnerability is lost or compromised, the generated data becomes ineffective for model training. Hence, we propose a novel vulnerability-preserving data augmentation method to generate new data

from the original dataset D. It primarily involves two steps. The first step (Section 3.3.1) is to slice all the statements related to the vulnerability. The second step (Section 3.3.2) is to augment the original dataset D by preserving the semantics of vulnerability-related statements and modifying the statements unrelated to the vulnerability.

3.3.1 Slicing Vulnerability-related Statements. Given a 4-tuple (f_v, f_p, S_{del}, S_{add}) from Section 3.2.2, we slice the statements that are related to S_{del} and S_{add}. To achieve this, we need to get the statement dependency relationship in a function. We utilize the program dependency graph (PDG) to obtain the data dependency and control dependency for each statement in f_v and f_p. The generated PDGs are defined as PDG_{f_v} and PDG_{f_p}. Based on PDG, we design an Algorithm 1 to slice the vulnerability-related statements. Specifically, for each statement s_{del}/s_{add} in S_{del}/S_{add}, a forward slicing procedure is performed in PDG to get a list of related statements S_f in the function f_v/f_p. This step is to ensure find out the *future* dependent statements from the current statement i.e., *start from current statement*. Then based on the obtained S_f, a backward slicing procedure is conducted to extract all relevant statements *before* the S_f i.e., *point to current statement*. Finally, we combine both directions for the added statements S_{add} and deleted statements S_{del} and obtain the vulnerability-related statements denoted as $S_{related}$.

3.3.2 Augmenting Code by Operators. As all vulnerability-related statements i.e., $S_{related}$ are obtained, we can augment the data on the vulnerability-unrelated statements from the vulnerable function f_v i.e., $\{s|s \in f_v \setminus S_{related}\}$ where s is the vulnerability-unrelated statement and $\setminus$ is the set difference operation between f_v and $S_{related}$. Preserving all vulnerability-related statements in the vulnerable function can retain its vulnerability and the proof is produced in Section 6.2. We define five types of mutation operations for vulnerable data augmentation, as shown in Table 2. Specifically, the operation *rn* means to rename the used variable names with all the occurrences of these variables with other names. The operation *ai* means to add a *if* condition which is the logical true before the assignment statements. For example, suppose an assignment statement *int a = b;* is vulnerability-unrelated, it can be transposed to *if (True) then int a = b;* after performing the *ai* operation. Operation *del* will randomly delete the statements that are not related to vulnerability-related statements, while the *add* will rename variable names in an assignment statement and add it back to the original function f_v, and the operation *ro* means to reorder the consequent assignment statements in the original function. By different types of mutation operators, we can greatly increase the amount of original data and enrich its diversity.

Algorithm 1: Vulnerability-related Slicing

```
Input: (PDG_fv, PDG_fp, S_del, S_add)
Output: S_related
Initialize S_related = set()
Function Slice(PDG_fv, PDG_fp, S_del, S_add):
    for s_del in S_del do
        S_f = traverse (s_del, PDG_fv, "forward")
        for s in S_f do
            S_b = traverse (s, PDG_fv, "backward")
            S_related = S_related ∪ S_b
    for s_add in S_add do
        S_f = traverse (s_add, PDG_fp, "forward")
        for s in S_f do
            S_b = traverse (s, PDG_fp, "backward")
            S_related = S_related ∪ S_b
Def traverse(s, pdg, direction):
    results = set(), visited = set()
    q = Queue()
    q.push(s)
    while q is not empty do
        u = q.pop()
        results.append(u)
        if direction == "forward" then
            S_f = the statements that start from u in pdg
        else if direction == "backward" then
            S_b = the statements that point to u in pdg
        for v in {S_f, S_b} do
            if v ∉ visited then
                visited.add(v)
                q.push(v)
    return results
```

Table 2. Mutation Operations for Data Augmentation.

Type	Definition
add	Rename identifiers in a assignment statement and add it back to the function.
ai	Add a *if* condition which is the logical truth before assignment statements.
rn	Rename the identifiers.
ro	Reorder assignment statements.
del	Delete statements that are not related to the vulnerability.

3.4 Edge-aware GGNN

While GGNN has found extensive application in modeling source code [1, 15, 33, 35, 52], it is noteworthy that the message passing is solely based on the node representations, i.e., $\boldsymbol{h}_{n_i}$, and the edge information is overlooked. We believe that the different types of edges in the Code Property Graph (CPG), such as "Flow to" and "Control" signify different semantics of programs, playing a crucial role in vulnerability detection. Building on this insight, we propose an edge-aware GGNN to leverage edge information effectively for vulnerability detection.

3.4.1 Graph Initialization. For both vulnerable and non-vulnerable functions, we utilize Joern [50] to obtain the Code Property Graph (CPG). In a formal representation, a

raw function c can be expressed as a multi-edged graph $g(\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of nodes, and $(v, u) \in \mathcal{E}$ denotes the edge connecting node v and node u. Each node possesses its node sequence, parsed by Joern from the original function. We tokenize the node sequence by spaces and punctuation. Additionally, for compound words (tokens constructed by concatenating multiple words according to camel or snake conventions), we split them into multiple tokens. We represent each token in the node sequence and each edge type connected with nodes using the learned embedding matrix $\boldsymbol{E}^{seqtoken}$ and $\boldsymbol{E}^{edgetype}$, respectively. Subsequently, the nodes and edges of the Code Property Graph (CPG) can be encoded as:

$$\begin{aligned} \boldsymbol{h}_v &= \text{SUM}(\boldsymbol{E}^{seqtoken}_{v,1}, ..., \boldsymbol{E}^{seqtoken}_{v,l}) \\ \boldsymbol{e}_{v,u} &= \boldsymbol{E}^{edgetype}_{v,u} \; if \; (v,u) \in \mathcal{E} \; else \; \boldsymbol{0} \end{aligned} \tag{3}$$

where l denotes the number of tokens in the node v. Hence, given the code property graph $g(\mathcal{V}, \mathcal{E})$, we have $\boldsymbol{H} \in \mathbb{R}^{m \times d}$, which denotes the initial node embedding matrix of the CPG, where m is the total number of nodes in the CPG and d is the dimension of the node embedding.

3.4.2 Edge-aware Message Passing. For every node v at each computation iteration k in the graph, we employ an aggregation function to calculate the aggregated vector $\boldsymbol{h}^k_{\mathcal{N}_{(v)}}$. This is achieved by considering a set of neighboring node embeddings, as well as the connected edge type information computed from the previous hop. As the edge information is also taken into account in the message passing process, it is specifically referred to as edge-aware message passing.

$$\boldsymbol{h}^k_{\mathcal{N}_{(v)}} = \text{SUM}(\{\text{Relu}(\boldsymbol{W}[\boldsymbol{h}^{k-1}_u; \boldsymbol{e}_{v,u}]) | \forall u \in \mathcal{N}_{(v)}\}) \tag{4}$$

where $\mathcal{N}_{(v)}$ is a set of the neighboring nodes which are directly connected with v, $\boldsymbol{W} \in \mathbb{R}^{(d+d') \times d}$ where d and d' are the dimension of the node and edge embedding, and Relu is the rectified linear unit [37]. For each node v, $\boldsymbol{h}^0_v$ is the initial node embedding of v, i.e., $\boldsymbol{h}_v \in \boldsymbol{H}$.

A Gated Recurrent Unit (GRU) [8] is used to update the node embeddings by incorporating the aggregation information.

$$\boldsymbol{h}^k_v = \text{GRU}(\boldsymbol{h}^{k-1}_v, \boldsymbol{h}^k_{\mathcal{N}_{(v)}}) \tag{5}$$

After n iterations of computation, we obtain the final node state $\boldsymbol{h}^n_v$ for node v. Subsequently, we apply max-pooling over all nodes $\boldsymbol{h}^n_v | \forall v \in \mathcal{V}$ to acquire the d-dimensional graph representation $\boldsymbol{h}^g$.

3.4.3 Classification Layer. After the message passing, we can get the graph representation $\boldsymbol{h}^g$ and use it for prediction. Specifically, a liner projection with a sigmoid activation function is used to make the final prediction.

$$y' = \text{Sigmoid}(\boldsymbol{W}'\boldsymbol{h}^g) \tag{6}$$

where y' is the logit produced by the sigmoid function and $\boldsymbol{W}' \in \mathbb{R}^{d \times 1}$ is the learned matrix.

3.5 Training

In *FGVulDet*, for each type of vulnerability (refer to Table 1), on the corresponding D_{train_i}, which is the training set containing vulnerable and non-vulnerable functions for the vulnerability type i, we train a set of binary classifiers $F = \{F_i, \forall i \in \text{Vul}\}$, where $\text{Vul} = \text{CWE} - \{404, 835, 120, 672, 362\}$ is the vulnerability type list to detect whether the function is vulnerable or not. The loss function for F_i is binary cross entropy.

$$l(y', y) = -(\cdot y \cdot \log(y') + (1 - y) * \log(1 - y')) \tag{7}$$

where y' is the logit (See Equation 6) and $y \in \{0, 1\}$ is the label with 1 for vulnerable and 0 otherwise. Totally, we have five classifiers according to different vulnerability types.

3.6 Testing

Since *FGVulDet* targets fine-grained vulnerability detection, we train each type of vulnerability as a single binary classifier F_i and vote to give the final prediction for a test sample. Specifically, given a function f_v (resp. f_p) from the test set, each type of classifier F_i is employed for detection $y'_i = F_i(f_v)$ and the predicted label can be expressed as follows:

$$\text{Predicted_label} = \begin{cases} 1 & y'_i > 0.5 \\ 0 & y'_i \leq 0.5 \end{cases} \tag{8}$$

where 1 for vulnerable and 0 for non-vulnerable. The final prediction result is determined through a majority voting mechanism from all classifiers based on the majority rule.

4 Evaluation Setup

- **RQ1**: What is the performance of *FGVulDet* compared with baselines in detecting vulnerable code?
- **RQ2**: Can each type of the defined mutation operations be beneficial to augment the training dataset to improve the detection accuracy?
- **RQ3**: What is the performance of our designed edge-aware GGNN compared with other GNN variants for vulnerability detection?

4.1 Dataset Details

The statistics for the five common vulnerability types on the collected dataset are presented in Table 3. We first collect a total of 92,525 commits of the five CWE types, then extract vulnerable and patched functions from each commit as vulnerable and non-vulnerable functions. After that, we extract the code property graph (CPG) for each function and obtain 165,222 graphs in total, which is less than the number of the raw functions due to the compilation errors of some functions with Joern [51]. We further conduct a data preprocessing to remove functions whose number of graph nodes is greater than 800, and finally obtain a raw data set with a total number of 99,076 functions. We divide the raw dataset into a train set, validation set, and test set

Table 3. The Statistics of the Collected DataSet.

CWE	Commit	Function	Graph	Preprocess	Raw dataset			Mutation
					train	validation	test	rn/del/add/ai/ro
CWE-404	39,261	78,522	67,860	41,816	25,060	8,400	8,356	12,552
CWE-835	16,584	33,168	29,904	16,105	9,638	3,263	3,204	4,839
CWE-120	10,877	21,754	19,800	11,187	6,710	2,250	2,227	3,370
CWE-672	7,906	15,812	15,006	8,689	5,197	1,741	1,751	2,604
CWE-362	17,897	35,794	32,652	21,279	12,768	4,247	4,264	6,390
Total	92,525	185,050	165,222	99,076	59,373	19,901	19,802	29,755

at a ratio of 6:2:2. In the end, we perform five types of mutation operations (see Table 2) to augment the vulnerable functions in the train set and generate the mutated functions for each type of mutation operations with a nearly equal amount of vulnerable functions. Note that, our dataset is more challenging than Devign [52]. In particular, the extracted non-vulnerable functions in Devign [52] come from non-vulnerable commits. However, *FGVulDet* uses the fixed version of the code from the vulnerability-related commits as the non-vulnerable functions for the model to learn. As the non-vulnerable functions are highly similar to the vulnerable functions in this operation compared with Devign, hence it is a more difficult data set for DL-based approaches to learn vulnerability features to distinguish them.

4.2 Baselines

We evaluate *FGVulDet* by comparing it against several well-known vulnerability detection approaches.

4.2.1 Static-analysis-based Approaches.

VUDDY [24]. It follows a process to abstract the function and then generates fingerprints for each function by hashing the normalized code. A target function is identified as vulnerable if its fingerprints match those of vulnerable functions.

MVP [49]. It is similar to VUDDY, which extracts vulnerability and patch signatures from a vulnerable function and its patched counterpart using a proposed program-slicing algorithm. Then It identifies a target function as vulnerable if it matches the vulnerability signature but does not match the patch signature.

4.2.2 Deep-learning-based Approaches.

Vuldeepecker [29]. It introduces a methodology involving extracting semantically connected statements associated with an argument of a library/API function call, forming code gadgets. Following program normalization, which standardizes user-defined variable names and function names, a bidirectional LSTM neural network is employed to determine the function's vulnerability.

Multi-Head Attention [44]. It has been widely used for modelling sequences. In particular, we leveraged the documentation from harvardnlp [26] to construct a multi-head attention layer, setting the number of heads to 4 and the maximum sequence length to 150 for comparative analysis.

Devign [52]. It is a typical work in vulnerability detection utilizing graph neural networks. Specifically, it combines varied semantics of a function into a unified graph structure to glean program semantics. Additionally, it employs a convolution module to capture features related to vulnerabilities.

CodeBERT [14]. It is a pre-trained model rooted in the Transformer architecture for code modeling. Leveraging millions of code data, it undergoes pre-training and subsequent fine-tuning for downstream code-related tasks. We have reproduced its implementation using the default settings provided in the official code on our dataset.

4.3 Experimental Settings

We utilized the common words where the frequency ≥ 3 from the training set, amounting to 90,000, to create the vocabulary set. The dimensions of word and edge embeddings were set to 128 respectively. A dropout of 0.3 was implemented after the word embedding layer. The hop value was set to 4 for CWE-404 and CWE-672, 1 for CWE-835, 5 for CWE-120, and 2 for CWE-362 to achieve optimal performance. We employed the Adam optimizer with an initial learning rate of 0.001 and a batch size of 64 for training. All experiments were conducted on the DGX server with three Nvidia Graphics Tesla V100.

5 Experimental Results

5.1 RQ1: Compared with Baselines

The experimental results are presented in Table 4. The first row presents the results for the static-analysis-based approaches, the second row is the results for DL-based approaches. The results for *FGVulDet* without/with the data augmentation are provided in the row of *FGVulDet* $_{none}$ and *FGVulDet* respectively.

When comparing the results of *FGVulDet* with static-analysis-based approaches, it is obvious that VUDDY achieves much higher precision scores. For instance, in the case of the vulnerability CWE-672 (Operation After Free), VUDDY achieves a precision score of 84.62 which is much higher than the score of *FGVulDet* 50.53. The higher precision score indicates that VUDDY has fewer false positive samples, which is reasonable as VUDDY relies on expert-crafted features to detect code vulnerabilities. These hand-crafted

Table 4. The experimental results of different approaches for vulnerability detection.

Approach	CWE-404			CWE-835			CWE-120			CWE-672			CWE-362		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
VUDDY	**73.17**	0.72	1.42	**70.00**	0.43	0.86	**79.41**	2.42	4.70	**84.62**	1.25	2.47	**60.00**	0.28	0.56
MVP	49.89	45.30	47.48	50.68	46.15	48.31	50.19	48.21	49.18	50.13	43.17	46.39	49.87	45.60	47.64
Vuldeepecker	67.26	38.19	48.71	50.70	58.52	54.33	50.55	49.51	50.02	51.39	56.95	54.02	51.57	43.64	47.28
Attention	56.82	57.66	57.24	50.94	65.22	57.21	55.1	57.91	56.47	52.79	52.73	52.76	51.38	53.75	52.54
Devign	65.86	40.98	50.52	50.79	61.50	55.64	55.30	63.90	59.29	51.56	43.17	46.99	51.16	44.01	47.32
CodeBERT	64.59	54.46	59.10	51.72	64.29	57.32	60.40	35.03	44.34	52.69	63.55	57.61	52.69	60.32	56.24
GGNN	64.96	46.01	53.87	51.64	59.70	55.38	55.71	60.14	57.84	51.67	60.02	55.53	53.49	55.01	54.24
GCN	65.17	40.07	49.63	50.82	76.88	61.19	55.72	64.88	59.95	52.99	60.59	56.54	52.76	54.68	53.71
FGVulDet$_{none}$	64.45	46.95	54.32	51.07	66.58	57.80	56.52	70.15	62.60	50.54	69.82	58.63	52.10	72.66	60.69
FGVulDet	53.91	**81.15**	**64.78**	50.51	**94.92**	**65.93**	52.93	**86.33**	**65.63**	50.53	92.26	65.30	50.47	**90.87**	**64.89**
FGVulDet$_{add}$	61.95	49.74	55.18	50.60	73.53	59.94	55.25	73.37	63.03	50.90	77.45	61.43	50.97	85.89	63.97
FGVulDet$_{ai}$	62.58	54.51	58.27	50.77	71.11	59.25	56.71	64.61	62.80	50.72	72.21	59.59	51.47	70.94	61.66
FGVulDet$_{del}$	60.80	58.40	59.57	51.47	62.86	56.60	56.28	66.49	60.96	52.48	55.47	53.93	51.57	67.26	58.38
FGVulDet$_{rn}$	53.41	78.31	63.51	50.49	91.94	65.19	53.20	83.82	65.09	50.45	**96.47**	**66.25**	51.03	87.80	64.54
FGVulDet$_{ro}$	60.38	58.71	59.54	51.41	76.81	61.60	55.32	67.83	60.94	50.18	77.45	60.90	52.05	63.30	57.12

vulnerability features are highly reliable by security experts. Therefore, if the samples being detected exhibit similar features, there is a high probability that they are vulnerable code. However, we can also find that VUDDY has a lower recall than *FGVulDet* i.e., 1.25 vs 92.26. It indicates that VUDDY has more false negative samples as these hand-crafted vulnerability features can only cover a limited number of vulnerability types, which leads to missing a substantial number of vulnerabilities compared to *FGVulDet*. In addition, we find that although MVP has lower precision scores than VUDDY, its recall scores are better than VUDDY, which indicates that MVP covers more vulnerability types but the detection precision is lower than VUDDY. Compared with these static-analysis-based approaches, *FGVulDet* is able to achieve a much higher recall, which leads to a higher F1-score.

When comparing the results of *FGVulDet* with the DL-based approaches, we can find that the pre-trained model CodeBERT performs better than other baselines in terms of F1. We speculate that the main reason is that CodeBERT uses extensive code-related data for pre-training and the model architecture is more powerful than the other baselines. Hence, CodeBERT has a stronger learning capability. However, *FGVulDet* outperforms it in terms of recall and F1. Even without data augmentation i.e., *FGVulDet $_{none}$* in Table 4, we can find that it still has better performance than CodeBERT in terms of F1 for vulnerability types CWE-835, 120, 672, 362), which indicates the effectiveness of our proposed approach.

Answer to RQ1: Although some static-analysis-based approaches have higher precision scores than *FGVulDet*, they have extensive false negative samples. Overall, in terms of Recall and F1, *FGVulDet* outperforms current baselines including static-analysis-based approaches and DL-based approaches by a significant margin.

5.2 RQ2: Effectiveness of Mutation Operations

In our work, we introduce five types of mutation operations to augment the data. We assess the effectiveness of each operation individually by conducting experiments using only one type of mutation operation at a time, while maintaining the hyper-parameters consistent with the original model. The results of these experiments are outlined in the final row of Table 4, where *FGVulDet$_*$* denotes the specific type of mutation operation being evaluated. The combined results of all five mutation operations are presented in the row labeled *FGVulDet*.

Through the analysis of the experimental results, it is evident that incorporating five types of mutation operations can significantly improve recall and F1 scores. For instance, in the case of vulnerability type CWE-404 (indicating memory leaks), *FGVulDet* improves recall and F1 from 46.95/54.32 to 81.15/64.78, respectively. Notably, the *rn* operation stands out as the most effective in enhancing F1 across different vulnerability types. Even for the vulnerability CWE-672, when fusing other mutation types of data to *rn*, F1 has a decrease. We conjuncture that *rn* operation appears to be efficient in improving the diversity of the training set compared to other mutation operations, making the model more robust and powerful.

Additionally, different types of mutations exhibit inconsistent performance compared to the original model without mutations (*FGVulDet$_{none}$*). For example, the mutation operations *add*, *ai*, and *rn* improve F1 for all types of vulnerabilities compared to *FGVulDet$_{none}$*. However, the *del* operation has a negative impact except for CWE-404 (memory leak), while *ro* improves F1 for vulnerability types CWE-{404, 835, 672} but has a negative impact for CWE-{120, 362} compared to *FGVulDet$_{none}$*. This may be attributed to the fact that the *del* and *ro* operations introduce some other types of vulnerabilities in the mutated functions, adding noise to the dataset and making it challenging for the model to make correct

decisions. For more details about the reason that *del* and *ro* can introduce new vulnerabilities, please refer to Section 6.2. Despite the negative impact of *del* and *ro* on specific vulnerability types, combining them with other mutation operations in *FGVulDet* yields the best overall performance.

Answer to RQ2: A thorough analysis of the performance of different mutation operations leads us to the conclusion that vulnerability-preserving data augmentation is effective for further enhancement.

5.3 RQ3: Effectiveness of Edge-aware GGNN

FGVulDet proposes the edge-aware GGNN which extends the current GGNN by encoding the edge-type information and using it in the message passing process to learn the vulnerability-related features. To illustrate the effectiveness of the proposed model, we also compared it with some GNN variants such as Gated Graph Network (GGNN) and Graph Convolution Network (GCN). The experimental results are shown in Table 4.

To make a fair comparison, we only compare the results of different GNN variants with $FGVulDet_{none}$. We find that compared with GGNN, supplementing the edge type information is beneficial for the model to detect vulnerabilities. It is reasonable as different types of vulnerabilities are involved in different aspects of the program i.e., in the different types of code property graph. For example, the vulnerability of CWE-120 (Buffer Overflow) is about the definition and usage of a variable, which can be captured in the program dependency graph via the edges of "Define/Use". However, the vulnerability of CWE-672 (Operation After Free) is related to the execution order of the statements and it is reflected in the control flow graph through the edges of "Flow to". Thus, we embed the edge type information explicitly and use it for learning will decrease the difficulty of the model in identifying the vulnerability and improve the model's capability. Furthermore, we also compare with GCN, which uses multiple graph convolutional layers in updating node representations, $FGVulDet_{none}$ still has a better performance excluding CWE-835 in terms of F1 score.

Answer to RQ3: Edge-aware GGNN, which encodes the edge type information explicitly for learning can increase the model capability and reduce the difficulty for the model to detect vulnerabilities, thus it can produce a better performance compared with some other GNN variants i.e., GGNN, GCN.

6 Discussion

6.1 Dataset Reliability

To ensure the reliability of our dataset, we established a systematic data collection pipeline. Initially, we crawled a substantial number of commits from 1614 C-language open-source projects on GitHub. Subsequently, we filtered out commits that only modified a single function and whose messages were matched by one vulnerability type (refer to Section 3.2). By employing this method, the functions extracted from the commits are more likely to be vulnerabilities with the correct vulnerability type. To validate the collected dataset, a team of professional security researchers randomly selected 400 commits from each of the 5 vulnerability types, totaling 2000 commits, and conducted a two-round cross-verification on the data labeling. The results revealed that 97.3% of commits were correctly classified (CWE-404 was 97.50%, CWE-835 was 96.50%, CWE-120 was 97.75%, CWE-672 was 98.50%, and CWE-362 was 96.25%, respectively). The high precision of our dataset indicates that vulnerability-related commits can be effectively identified and correctly classified using keywords from an unlabeled dataset.

6.2 Proof of Vulnerability-preserving

Here we discuss the proof of vulnerability-preserving augmentation, which can preserve the vulnerability of a function after being modified. First, we have the following notation S_{vul} denotes the statements that trigger the vulnerability in a function, and $S_{vul_{dep}}$ denotes the dependent statements of the statements S_{vul} in a function. We have the assumption:
Assumption: If all S_{vul} and $S_{vul_{dep}}$ are retained after the mutation operations, then the vulnerability still exists.

The assumption is correct since the vulnerabilities are composed of S_{vul} and $S_{vul_{dep}}$. Hence, as long as we prove that our proposed method retains all S_{vul} and $S_{vul_{dep}}$, it is vulnerability-preserving.

We understand that commits in GitHub primarily serve to record code changes for different versions, and the commits extracted through our Data Collection process (Section 3.2) are identified as vulnerability-related through keyword filtering. The cross-validation results presented in Section 6.1 confirm the reliability of these commits concerning vulnerabilities. Consequently, the changed statements S_{del} and S_{add} in the vulnerability commit are unequivocally associated with the vulnerability, i.e., $\exists S \subseteq S_{del} \cup S_{add} \Rightarrow S \subseteq S_{vul} \cup S_{vul_{dep}}$. This suggests that if we apply the vulnerability-related slicing algorithm 1 by iteratively searching for related statements via PDG to each statement in S_{del} and S_{add} separately, then all $S_{vul} \cup S_{vul_{dep}}$ can be encompassed, i.e.,

$$\{S_{vul} \cup S_{vul_{dep}}\} \subseteq S_{related} \tag{9}$$

Once we obtain the vulnerability-related statements $S_{related}$, we can proceed with data augmentation by considering the set difference between the raw vulnerable function f_v and $S_{related}$. This augmentation process ensures the preservation of vulnerability in the augmented functions.

However, it is important to acknowledge that the defined mutation operations cannot guarantee the absence of new

vulnerabilities. Consider the scenario where a statement like "usbDevs = NULL;" is unrelated to vulnerabilities and is used to set the pointer "usbDevs" to NULL to mitigate the vulnerability of "use after free". If we delete the statement "usbDevs = NULL;" using the mutation operation *del*, it could introduce the vulnerability CWE-672 (Operation After Free). Similarly, in the case of a shared variable among multiple threads, if its assignment statement is placed before the lock operation, using the mutated operation *ro* might result in a vulnerability CWE-362 (race condition). Fortunately, we observe that the proposed operations *rn*, *add*, and *ai* do not introduce new vulnerabilities. Only two operations, *del*, and *ro*, have the potential to generate new vulnerabilities. Moreover, as indicated in Section 5.2, the combination of all five mutation operations in *FGVulDet* yields the highest performance. In summary, ensuring the absence of new vulnerabilities when modifying the semantics of the original function is a significant challenge, and addressing this remains in future.

6.3 Threats to Validity

The first potential threat concerns the limited scope of the collected dataset, which includes only five types of vulnerabilities in the C programming language. While this might not cover all possible vulnerabilities, we assert that the selected vulnerability types are common and capable of causing significant harm to software systems. Moreover, the proposed model is not restricted to this specific dataset and can be extended to detect other types of vulnerabilities. Another potential threat is the exclusion of functions with node sizes exceeding 800 in the graph. This practice is consistent with Devign [52] due to the GPU memory requirements of GNNs. Truncating the graph size is a pragmatic choice to facilitate experiments, as GNNs necessitate substantial GPU memory.

7 Related Work

Vulnerability Detection Static analysis plays a crucial role in identifying flaws and errors in a codebase to prevent the introduction of vulnerabilities and security bugs. Commercial tools like flawfinder [48] and CPPCheck [3] provide extensive static code analysis, helping discover early bugs and vulnerabilities. VUDDY utilizes a clone-based approach by matching the signature of vulnerable functions with target program signatures. Another approach by Yang et al. [49] employs novel slicing techniques while incorporating vulnerability signatures. However, these analyses often require significant context and the extensive efforts of experts. With the increasing popularity of deep learning approaches, various works, such as VulDeePecker [29], VulSniper [12], and Devign [52], have utilized deep learning techniques to predict and detect vulnerabilities. Saikat et al. [5] systematically study the existing deep-learning-based vulnerability detection approaches. Besides the function-level vulnerability detection, LineVD [22] leverages graph neural networks to locate the buggy statements. LineVul [16] employs the transformer to identify the line vulnerability. These methods typically use an intermediate representation of programs, such as tokens or graphs, to facilitate the learning of meaningful program representations. In contrast, our work introduces a vulnerability-preserving data augmentation process to enhance vulnerability data for training purposes, which offers a different perspective.

Data Augmentation It is a valuable technique for expanding datasets during model training in various domains such as computer vision (CV), natural language processing (NLP), and automated speech recognition (ASR). This technique is particularly useful in domains with limited datasets to prevent overfitting, as seen in medical image analysis [41]. Moreover, data augmentation has been employed to enhance model performance. Studies [13, 39] introduce new methods for data augmentation, aiming to achieve better performance in image classification tasks. In code representation, Jain et al. [23] and Liu et al. [34] explore data augmentation in pre-training. Zhuo et al. [53] and Dong et al. [10] conduct the literature review of source code augmentation. The current strategies used are semantic-preserving at the functional granularity, which may not preserve vulnerability when generating mutations for code vulnerabilities. In contrast, our work introduces vulnerability-preserving data augmentation. Our approach augments the original dataset by preserving the semantics of vulnerability-related statements while modifying statements unrelated to vulnerabilities. It ensures that the generated variants retain vulnerability information, distinguishing it from other semantic-preserving strategies.

8 Conclusion

In this paper, we introduce a fine-grained vulnerability detector namely *FGVulDet*, which employs multiple classifiers to learn characteristics of various vulnerability types for source code vulnerability detection. To address the scarcity of data for some vulnerability types, we propose a novel vulnerability-preserving data augmentation technique for augmentation. Furthermore, we extend GGNN to an edge-aware variant to capture edge-type information. Extensive experiments have confirmed the effectiveness of *FGVulDet*.

Acknowledgments

This research is supported by the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN), the National Research Foundation, Singapore, and DSO National Laboratories under the AI Singapore Programme (AISG Award No: AISG2-GC-2023-008), and NRF Investigatorship NRF-NRFI06-2020-0001. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore.

References

[1] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to Represent Programs with Graphs. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. https://openreview.net/forum?id=BJOFETxR-

[2] Domagoj Babic, Lorenzo Martignoni, Stephen McCamant, and Dawn Song. 2011. Statically-directed dynamic automated test generation. In *Proceedings of the 20th International Symposium on Software Testing and Analysis, ISSTA 2011, Toronto, ON, Canada, July 17-21, 2011*, Matthew B. Dwyer and Frank Tip (Eds.). ACM, 12–22. https://doi.org/10.1145/2001420.2001423

[3] CERN. 2007. *CPPCheck*. http://cppcheck.sourceforge.net/

[4] Sang Kil Cha, Maverick Woo, and David Brumley. 2015. Program-Adaptive Mutational Fuzzing. In *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*. IEEE Computer Society, 725–741. https://doi.org/10.1109/SP.2015.50

[5] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Trans. Software Eng.* 48, 9 (2022), 3280–3296. https://doi.org/10.1109/TSE.2021.3087402

[6] Hongxu Chen, Yinxing Xue, Yuekang Li, Bihuan Chen, Xiaofei Xie, Xiuheng Wu, and Yang Liu. 2018. Hawkeye: Towards a Desired Directed Grey-box Fuzzer. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM, 2095–2108. https://doi.org/10.1145/3243734.3243849

[7] Xiao Cheng, Haoyu Wang, Jiayi Hua, Miao Zhang, Guoai Xu, Li Yi, and Yulei Sui. 2019. Static Detection of Control-Flow-Related Vulnerabilities Using Graph Embedding. In *24th International Conference on Engineering of Complex Computer Systems, ICECCS 2019, Guangzhou, China, November 10-13, 2019*, Jun Pang and Jing Sun (Eds.). IEEE, 41–50. https://doi.org/10.1109/ICECCS.2019.00012

[8] Kyunghyun Cho, Bart van Merrienboer, Çaglar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*, Alessandro Moschitti, Bo Pang, and Walter Daelemans (Eds.). ACL, 1724–1734. https://doi.org/10.3115/V1/D14-1179

[9] Hoa Khanh Dam, Truyen Tran, Trang Pham, Shien Wee Ng, John Grundy, and Aditya Ghose. 2017. Automatic feature learning for vulnerability prediction. *CoRR* abs/1708.02368 (2017). arXiv:1708.02368 http://arxiv.org/abs/1708.02368

[10] Zeming Dong, Qiang Hu, Yuejun Guo, Zhenya Zhang, and Jianjun Zhao. 2023. Boosting Source Code Learning with Text-Oriented Data Augmentation: An Empirical Study. In *23rd IEEE International Conference on Software Quality, Reliability, and Security, QRS 2023 Companion, Chiang Mai, Thailand, October 22-26, 2023*. IEEE, 383–392. https://doi.org/10.1109/QRS-C60940.2023.00017

[11] Xiaoning Du, Bihuan Chen, Yuekang Li, Jianmin Guo, Yaqin Zhou, Yang Liu, and Yu Jiang. 2019. Leopard: identifying vulnerable code for vulnerability assessment through program metrics. In *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019*, Joanne M. Atlee, Tevfik Bultan, and Jon Whittle (Eds.). IEEE / ACM, 60–71. https://doi.org/10.1109/ICSE.2019.00024

[12] Xu Duan, Jingzheng Wu, Shouling Ji, Zhiqing Rui, Tianyue Luo, Mutian Yang, and Yanjun Wu. 2019. VulSniper: Focus Your Attention to Shoot Fine-Grained Vulnerabilities. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, Sarit Kraus (Ed.). ijcai.org, 4665–4671. https://doi.org/10.24963/IJCAI.2019/648

[13] Alhussein Fawzi, Horst Samulowitz, Deepak S. Turaga, and Pascal Frossard. 2016. Adaptive data augmentation for image classification. In *2016 IEEE International Conference on Image Processing, ICIP 2016, Phoenix, AZ, USA, September 25-28, 2016*. IEEE, 3688–3692. https://doi.org/10.1109/ICIP.2016.7533048

[14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL, Vol. EMNLP 2020)*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, 1536–1547. https://doi.org/10.18653/V1/2020.FINDINGS-EMNLP.139

[15] Patrick Fernandes, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Structured Neural Summarization. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net. https://openreview.net/forum?id=H1ersoRqtm

[16] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A Transformer-Based Line-Level Vulnerability Prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories* (Pittsburgh, Pennsylvania) *(MSR '22)*. Association for Computing Machinery, New York, NY, USA, 608–620. https://doi.org/10.1145/3524842.3528452

[17] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017 (Proceedings of Machine Learning Research, Vol. 70)*, Doina Precup and Yee Whye Teh (Eds.). PMLR, 1263–1272. http://proceedings.mlr.press/v70/gilmer17a.html

[18] ChengYue Gong, Tongzheng Ren, Mao Ye, and Qiang Liu. 2020. MaxUp: A Simple Way to Improve Generalization of Neural Network Training. *CoRR* abs/2002.09024 (2020). arXiv:2002.09024 https://arxiv.org/abs/2002.09024

[19] William L. Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 1024–1034. https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebea9-Abstract.html

[20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 770–778. https://doi.org/10.1109/CVPR.2016.90

[21] Dan Hendrycks, Norman Mu, Ekin Dogus Cubuk, Barret Zoph, Justin Gilmer, and Balaji Lakshminarayanan. 2020. AugMix: A Simple Data Processing Method to Improve Robustness and Uncertainty. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net. https://openreview.net/forum?id=S1gmrxHFvB

[22] David Hin, Andrey Kan, Huaming Chen, and M. Ali Babar. 2022. LineVD: Statement-Level Vulnerability Detection Using Graph Neural Networks. In *Proceedings of the 19th International Conference on Mining Software Repositories* (Pittsburgh, Pennsylvania) *(MSR '22)*. Association for Computing Machinery, New York, NY, USA, 596–607. https://doi.org/10.1145/3524842.3527949

[23] Paras Jain, Ajay Jain, Tianjun Zhang, Pieter Abbeel, Joseph E Gonzalez, and Ion Stoica. 2020. Contrastive Code Representation Learning. *arXiv preprint arXiv:2007.04973* (2020).

[24] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. Vuddy: A scalable approach for vulnerable code clone discovery. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 595–614.

[25] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[26] Guillaume Klein, Yoon Kim, Yuntian Deng, Jean Senellart, and Alexander M. Rush. 2017. OpenNMT: Open-Source Toolkit for Neural Machine Translation. In *Proc. ACL*. https://doi.org/10.18653/v1/P17-4012

[27] Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2015. Gated graph sequence neural networks. *arXiv preprint arXiv:1511.05493* (2015).

[28] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2021. SySeVR: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* (2021).

[29] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681* (2018).

[30] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, and Yuyi Zhong. 2018. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In *25th Annual Network and Distributed System Security Symposium (NDSS 2018)* (San Diego, California, USA).

[31] Guanjun Lin, Jun Zhang, Wei Luo, Lei Pan, Olivier De Vel, Paul Montague, and Yang Xiang. 2019. Software vulnerability discovery via learning multi-domain knowledge bases. *IEEE Transactions on Dependable and Secure Computing* (2019).

[32] Shangqing Liu. 2020. A Unified Framework to Learn Program Semantics with Graph Neural Networks. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1364–1366.

[33] Shangqing Liu, Yu Chen, Xiaofei Xie, Jingkai Siow, and Yang Liu. 2020. Retrieval-augmented generation for code summarization via hybrid gnn. *arXiv preprint arXiv:2006.05405* (2020).

[34] Shangqing Liu, Bozhi Wu, Xiaofei Xie, Guozhu Meng, and Yang Liu. 2023. Contrabert: Enhancing code pre-trained models via contrastive learning. *arXiv preprint arXiv:2301.09072* (2023).

[35] Shangqing Liu, Xiaofei Xie, Jingkai Siow, Lei Ma, Guozhu Meng, and Yang Liu. 2023. Graphsearchnet: Enhancing gnns via capturing global dependencies for semantic code search. *IEEE Transactions on Software Engineering* (2023).

[36] David Z. Morris. 2017. *How Equifax Turned Its Massive Hack Into an Even Worse Dumpster Fire?* http://fortune.com/2017/09/09/equifax-hack-crisis/

[37] Vinod Nair and Geoffrey E Hinton. 2010. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*. 807–814.

[38] Van-Anh Nguyen, Dai Quoc Nguyen, Van Nguyen, Trung Le, Quan Hung Tran, and Dinh Phung. 2022. ReGVD: Revisiting graph neural networks for vulnerability detection. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 178–182.

[39] Luis Perez and Jason Wang. 2017. The effectiveness of data augmentation in image classification using deep learning. *arXiv preprint arXiv:1712.04621* (2017).

[40] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated Vulnerability Detection in Source Code Using Deep Representation Learning. In *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 757–762.

[41] Hoo-Chang Shin, Neil A Tenenholtz, Jameson K Rogers, Christopher G Schwarz, Matthew L Senjem, Jeffrey L Gunter, Katherine P Andriole, and Mark Michalski. 2018. Medical image synthesis for data augmentation and anonymization using generative adversarial networks. In *International workshop on simulation and synthesis in medical imaging*. Springer, 1–11.

[42] Nick Stephens, John Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2016. Driller: Augmenting Fuzzing Through Selective Symbolic Execution.. In *NDSS*, Vol. 16. 1–16.

[43] Julien Vanegue and Shuvendu K Lahiri. 2013. Towards practical reactive security audit using extended static checkers. In *2013 IEEE Symposium on Security and Privacy*. IEEE, 33–47.

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.

[45] John Viega, Jon-Thomas Bloch, Yoshi Kohno, and Gary McGraw. 2000. ITS4: A static vulnerability scanner for C and C++ code. In *Proceedings 16th Annual Computer Security Applications Conference (ACSAC'00)*. IEEE, 257–267.

[46] Huanting Wang, Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Yansong Feng, Lizhong Bian, and Zheng Wang. 2020. Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Transactions on Information Forensics and Security* 16 (2020), 1943–1958.

[47] Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. 2017. Skyfire: Data-driven seed generation for fuzzing. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 579–594.

[48] David A. Wheeler. 2017. *Flawfinder*. https://www.dwheeler.com/flawfinder/

[49] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, et al. 2020. {MVP}: Detecting Vulnerabilities using Patch-Enhanced Vulnerability Signatures. In *29th {USENIX} Security Symposium ({USENIX} Security 20)*. 1165–1182.

[50] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE Symposium on Security and Privacy*. IEEE, 590–604.

[51] Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy (SP '14)*. IEEE Computer Society, Washington, DC, USA, 590–604. https://doi.org/10.1109/SP.2014.44

[52] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, and Yang Liu. 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. In *Advances in Neural Information Processing Systems*. 10197–10207.

[53] Terry Yue Zhuo, Zhou Yang, Zhensu Sun, Yufei Wang, Li Li, Xiaoning Du, Zhenchang Xing, and David Lo. 2023. Data Augmentation Approaches for Source Code Models: A Survey. *arXiv preprint arXiv:2305.19915* (2023).

[54] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2019. μVulDeePecker: A deep learning-based system for multiclass vulnerability detection. *IEEE Transactions on Dependable and Secure Computing* (2019).